@assistant

Appears in: **DL-06**

agents/assistant.py:21-22, 78-180, 183-194

> *"Single-pass ReAct routing: one LLM call decides which agents re-run, then they run in pipeline order." … "After each /refine, get_routing returns agents_to_run, reasoning, and routing_label, and the label renders in the panel (@agent-contracts)."*

@assistant

agents/assistant.py:21-22

```python
# Pipeline order matters — agents are always run in this sequence
ALL_AGENTS = ["audience", "copywriter", "guardrails", "cta_optimizer"]
```

…

agents/assistant.py:78-180

```python
def get_routing(
    current_output: dict,
    user_message: str,
    tone_val: int,
    age_val: int,
    formality_val: int,
    chat_history: list,
) -> dict:
    """
    ReAct reasoning step: determine which pipeline agents need to re-run
    based on user feedback and slider changes.

    Returns:
        {
            "agents_to_run": list[str],  # ordered subset of ALL_AGENTS
            "reasoning": str,
            "routing_label": str,        # e.g. "Planner → Copywriter → CTA → Output"
        }
    """
    try:
        # Detect which sliders moved from their defaults (3 / 35 / 3)
        slider_changes = []
        if tone_val != 3:
            slider_changes.append(f"tone changed to {TONE_MAP.get(tone_val, tone_val)}")
        if age_val != 35:
            slider_changes.append(f"audience age shifted to ~{age_val}")
        if formality_val != 3:
            slider_changes.append(f"formality changed to level {formality_val}/5")

        history_str = ""
        if chat_history:
            recent = chat_history[-3:]
            history_str = "\n".join(
                f"{m['role'].title()}: {m['content']}" for m in recent
            )

        system_prompt = (
            "You are a marketing pipeline router. Based on the user's request and "
            "slider changes, decide which agents to re-run. Apply these rules:\n"
            "- Tone or formality changes → include: copywriter, guardrails, cta_optimizer\n"
            "- Age range change → include: audience, copywriter, guardrails, cta_optimizer\n"
            "- User mentions ad copy, wording, text, writing style → include: copywriter, guardrails, cta_optimizer\n"
            "- User mentions CTA, call to action, button text → include: cta_optimizer\n"
            "- User mentions audience, persona, demographics, who → include: audience, copywriter, guardrails, cta_optimizer\n"
            "- User mentions product, image, visual, tags → include: audience, copywriter, guardrails, cta_optimizer\n"
            "- If unclear → default to: copywriter, guardrails, cta_optimizer\n\n"
            "Valid agent names: audience, copywriter, guardrails, cta_optimizer\n"
            "Respond ONLY with valid JSON (no markdown fences):\n"
            '{"agents_to_run": [...], "reasoning": "1-2 sentence explanation", '
            '"routing_label": "Planner → AgentA → AgentB → Output"}'
        )

        user_prompt = f"User message: {user_message or '(none — slider-only change)'}\n"
        if slider_changes:
            user_prompt += f"Slider changes: {', '.join(slider_changes)}\n"
        if history_str:
            user_prompt += f"Recent chat:\n{history_str}\n"

        payload = {
            "model": OLLAMA_MODEL,
            "stream": False,
            "messages": [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt},
            ],
        }

        resp = requests.post(f"{OLLAMA_URL}/api/chat", json=payload, timeout=120)
        resp.raise_for_status()
        content = resp.json()["message"]["content"].strip()

        if content.startswith("```"):
            content = content.split("```")[1]
            if content.startswith("json"):
                content = content[4:]

        parsed = json.loads(content.strip())

        # Validate agent names against whitelist
        valid = [a for a in parsed.get("agents_to_run", []) if a in ALL_AGENTS]
        if not valid:
            valid = ["copywriter", "guardrails", "cta_optimizer"]

        # Enforce pipeline order
        valid = [a for a in ALL_AGENTS if a in valid]

        # guardrails must always follow copywriter
        if "copywriter" in valid and "guardrails" not in valid:
            idx = valid.index("copywriter")
            valid.insert(idx + 1, "guardrails")

        return {
            "agents_to_run": valid,
            "reasoning": parsed.get("reasoning", "Running default copy refresh."),
            "routing_label": parsed.get("routing_label", "Planner → Copywriter → Output"),
        }

    except Exception:
        return {
            "agents_to_run": ["copywriter", "guardrails", "cta_optimizer"],
            "reasoning": "Running default copy refresh.",
            "routing_label": "Planner → Copywriter → CTA → Output",
        }
```

…

agents/assistant.py:183-194

```python
def map_sliders_to_params(tone_val: int, age_val: int, formality_val: int) -> dict:
    """Convert raw slider integers to agent-ready parameter strings."""
    tone = TONE_MAP.get(tone_val, "professional")
    age_hint = f"{max(18, age_val - 7)}–{min(65, age_val + 7)}"
    formality_instruction = FORMALITY_MAP.get(
        formality_val, "balanced, professional but approachable"
    )
    return {
        "tone": tone,
        "age_hint": age_hint,
        "formality_instruction": formality_instruction,
    }
```

What it shows: The ReAct router: a single LLM call picks the agent subset, which is then forced into pipeline order with guardrails always following the copywriter. `map_sliders_to_params` turns slider ints into tone/age/formality strings.

Source: agents/assistant.py:21-22, 78-180, 183-194 · image rendered by build_snippets.py